ENTERPRISE TECHNICAL WHITEPAPER & SYSTEM MANUAL

DATABASE ARCHITECTURE & OBJECT PERSISTENCE ANALYSIS

Complete Specification of the Dual-Layer Hybrid Database Engine, Relational Schemas, Object Lifecycles, and Native Google Sheets Synchronization

ডিম ব্যবসার ডিজিটাল হালখাতা ডাটাবেজ আর্কিটেকচার ও অবজেক্ট সংরক্ষণ পদ্ধতি — পূর্ণাঙ্গ প্রযুক্তিগত দলিল

APPLICATION NAME
YolkFlow™ (Next.js 16.3 + React 19)

PRIMARY CLOUD MASTER
Google Sheets API v4 (JWT Service Account)

LOCAL PERSISTENCE LAYER
mock_db.json (Zero-Latency Atomic Disk Cache)

OBJECT LIFECYCLE ENGINE
Two-Phase Idempotent Write & Native Formula Cloner

HYBRID DATA FORMAT
Relational 2D Rows + Embedded JSON Arrays

SECURITY & RBAC
Role-Based Granular Permissions (Admin, Manager, Viewer)

Document Version 2.0 • Deep Codebase Analysis (src/lib/db.ts & src/lib/googleSheets.ts)

6 High-Fidelity Architectural Vector Diagrams Included

Table of Contents

This document presents an exhaustive, engineering-grade breakdown of how data is structured, modeled, serialized, cached, synchronized, and queried within the YolkFlow egg trading system.

Chapter 1: Executive Summary & Dual-Layer Database Philosophy Page 3

Chapter 2: High-Level Storage Architecture & Service Layer Page 4

• *Diagram 1: Dual-Layer Hybrid Storage & Cloud Sync Architecture* Page 5

Chapter 3: Entity-Relationship Modeling & Data Schema Dictionary Page 6

• *Diagram 2: Relational Entity-Relationship Diagram (ERD)* Page 7

Chapter 4: The Complete Object Saving Lifecycle (End-to-End Walkthrough) Page 8

• *Diagram 3: Object Saving Lifecycle UML Sequence Pipeline* Page 9

Chapter 5: Data Serialization, Deserialization & Hybrid Storage Page 10

Chapter 6: Cloned Daily Tab: Physical Cell Grid & Native Formula Engine Page 11

• *Diagram 4: Physical Spreadsheet Grid Layout & Formula Mapping* Page 12

Chapter 7: Read Pipeline, Hydration & Cache Merging Engine Page 13

• *Diagram 5: Read Pipeline, Data Hydration & Cache Merge Flow* Page 14

Chapter 8: Overhead Expenses, Savings Funds & Samity Cross-Sync Page 15

• *Diagram 6: Overhead, Savings & Samity Cross-Sync Architecture* Page 16

Chapter 9: Concurrency, Atomicity & ACID Transaction Analysis Page 17

Chapter 10: Performance Benchmarks, API Quotas & Scalability Limits Page 18

Chapter 11: Enterprise Scaling Blueprint: Migration to PostgreSQL / Supabase Page 19

Chapter 12: Code Deep-Dive: Implementation Walkthrough Page 20

Appendix: Data Contracts, JSON Schemas & Bengali Accounting Glossary Page 21

> **Direct Source Code Lineage:** Every schema, pipeline step, and formula documented herein directly reflects the TypeScript implementation inside `d:\eggapp\src\lib\db.ts`, `src\lib\googleSheets.ts`, `src\lib\ledger.ts`, `src\app\actions.ts`, and related API routes.

Chapter 1: Executive Summary & Dual-Layer Database Philosophy

1.1 The Architectural Challenge: Digitizing the Hal-Khata

Wholesale egg merchandising in Bangladesh (e.g. M.A. Khalek Sarker Wholesale Depot) operates under unique commercial constraints. Merchants rely on physical paper accounting notebooks (**দৈনিক হালখাতা**) where four interconnected tables balance inventory valuation, customer credit collections ("বাকি"), supplier payables ("মহাজন দেনা"), and operating expenses.

Standard enterprise relational databases (such as pure PostgreSQL or MySQL) present two immediate operational friction points for this domain:

1. **Loss of Direct Human Auditability:** Non-technical business owners, accountants, and partners must be able to open their accounting records on a mobile phone, see familiar tabular columns, and audit formulas without writing SQL queries.
2. **Depot Connectivity Vulnerability:** Wholesale physical markets frequently suffer from network drops. If an internet failure prevents an accountant from recording the evening closing cash and egg stock, the application becomes unusable.

1.2 The YolkFlow Dual-Layer Hybrid Database Solution

To resolve both friction points simultaneously, YolkFlow employs a **Dual-Layer Hybrid Persistence Architecture**:

CLOUD MASTER	LOCAL ENGINE	DYNAMIC TABS
Google Sheets API	**mock_db.json**	**Cloned Daily Tabs**
Human-auditable cloud master with native formulas	Atomic zero-latency disk cache with offline immunity	DD/MM/YY cloned sheets with live Excel formulas

Under this paradigm, the system writes to the local JSON database first, ensuring zero data loss regardless of internet health. It then synchronizes asynchronously with Google Sheets API v4 using a Google Cloud Service Account with JWT authentication. If Google Cloud is unreachable, permissions are misconfigured, or API limits are hit, the application gracefully continues uninterrupted operations from the local cache.

1.3 Core Tenets of the Database Engine

- **Idempotent Upsert Strategy:** Saving an entry for an existing date never creates duplicate rows; it cleanses previous entries for that date and injects the updated snapshot chronologically.
- **Hybrid Relational + JSON Storage:** Flat tabular rows represent structured scalars, while nested variable-length arrays (e.g. itemized extra collections and customer dues) are serialized into JSON text within designated spreadsheet columns.
- **Native Formula Preservation:** Rather than writing dead static numbers into Google Sheets, the engine injects live spreadsheet formulas (`=B6*C6` , `=SUM(B6:B11)` , `=B44-E24`) so calculations can be audited and cross-referenced in standard spreadsheet tools.

Chapter 2: High-Level Storage Architecture & Service Layer

2.1 The Four Tiers of Data Flow

The YolkFlow persistence tier is structured into four distinct physical and logical layers:

- **Tier 1: Client Browser State:** React 19 Client Components manage reactive form state in memory, backed by browser `localStorage` for user authentication tokens and role sessions.
- **Tier 2: Next.js Server & Orchestration Layer (`src/lib/db.ts`):** Next.js Server Actions (`actions.ts`) receive strongly-typed payloads. `db.ts` coordinates the dual-write pipeline, validates payloads, and handles error recovery.
- **Tier 3: Local Persistence Layer (`mock_db.json`):** A structured JSON document persisted directly on the server filesystem (`process.cwd() + "/mock_db.json"`). Provides sub-5ms read and write operations.
- **Tier 4: Google Sheets Cloud Master:** Cloud spreadsheet hosted on Google Drive, manipulated via the official `googleapis` SDK (v176.0.0). Organizes data into master logs and cloned daily sheets.

2.2 Google Cloud Service Account Authentication (JWT)

Authentication to the Google Sheets API is established using a headless Service Account configured in Google Cloud Console. The authentication client is initialized in `src/lib/googleSheets.ts` :

```
const auth = new google.auth.GoogleAuth({
  credentials: {
    client_email: process.env.GOOGLE_SERVICE_ACCOUNT_EMAIL,
    private_key: getPrivateKey(), // Handles Vercel escaped newlines (\n)
  },
  scopes: ["https://www.googleapis.com/auth/spreadsheets"],
});
sheetsClient = google.sheets({ version: "v4", auth });
```

> **Vercel Private Key Formatting:** When deploying to Vercel or cloud environments, RSA private keys with embedded `\n` newline characters are automatically sanitized by `getPrivateKey()` using `key.replace(/\\n/g, "\n")` to prevent cryptographic decoding failures.

2.3 High-Level System Vector Diagram

The following diagram maps the complete end-to-end topology across all four operational tiers.



DailyStock (Cols A-H) • **Financials** (Cols A-N) • **Expenses** (Cols A-G) • **ProviderDues** (Cols A-H) Overhead & Savings Tabs • **OverheadExpenses** (Cols A-I) Auto-synced with daily সমিতি • **SavingsAndExtras** (Cols A-J) Cloned Daily Tabs (DD/MM/YY) • Cloned from `Original` • Bengali 4-table Hal-Khata • **Native Formulas Injected:** =B6*C6, =B44-E24, =SUM(..) Auditability Benefits ✓ Mobile app accessible by owner ✓ Built-in Google Drive version history ✓ Zero monthly cloud database bill ✓ Native spreadsheet export to XLSX/PDF RPC Step 1 Step 2 (Parallel Sheets API v4) Fallback

FIGURE 2.1: DUAL-LAYER HYBRID STORAGE & CLOUD SYNCHRONIZATION ARCHITECTURE

Chapter 3: Entity-Relationship Modeling & Data Schema Dictionary

3.1 Relational Schema Implemented on Flat Storage

Although Google Sheets is fundamentally a flat two-dimensional grid, YolkFlow structures data into an explicit relational schema where the ISO Date string (`YYYY-MM-DD`) functions as the primary temporal foreign key connecting master entities.

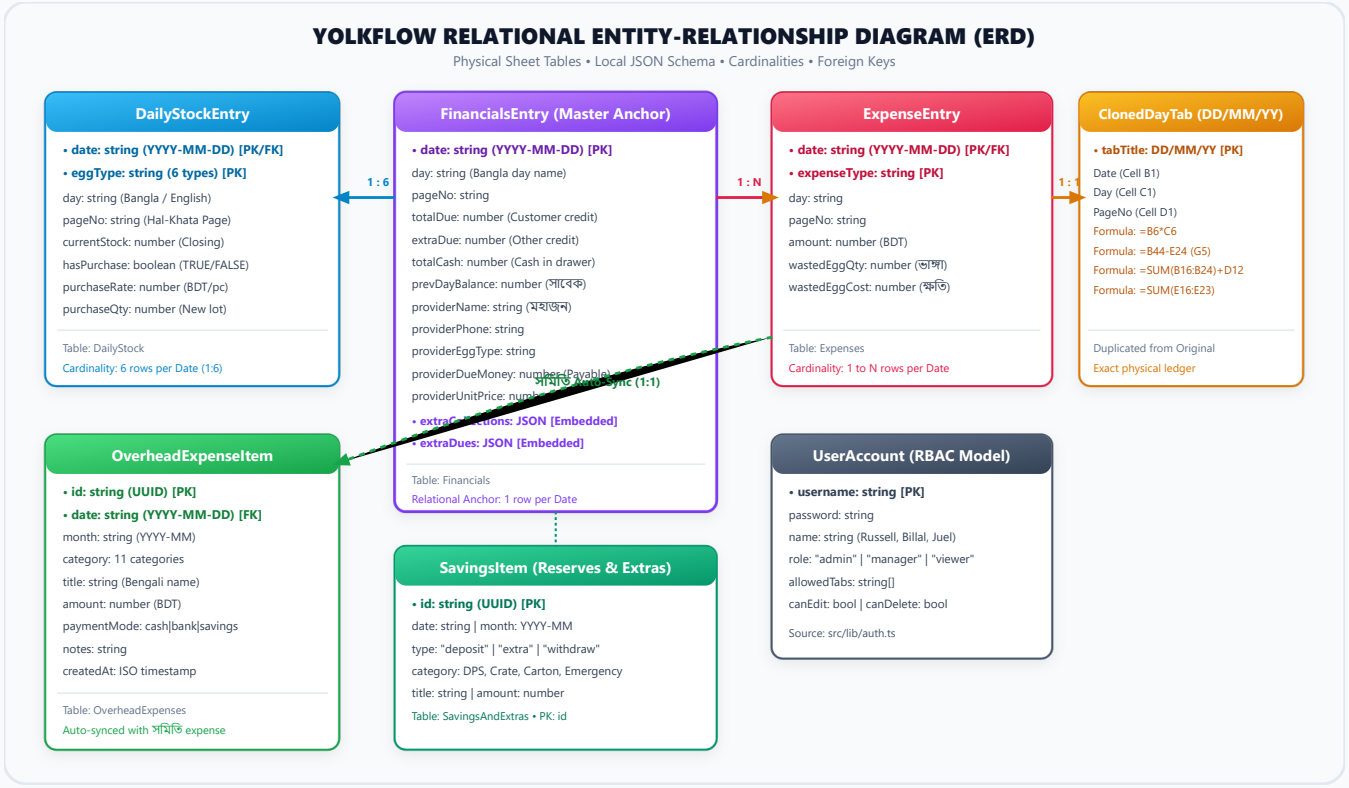


FIGURE 3.1: RELATIONAL ENTITY-RELATIONSHIP DIAGRAM (ERD) & SCHEMA MODELS

3.2 Master Schema Specifications

1. DailyStockEntry (Worksheet: `DailyStock`)

Column	Field Name	TypeScript Type	Constraint / Notes
A	Date	`string`	ISO Date (YYYY-MM-DD), Composite PK
B	Day	`string`	Bangla / English day name (e.g. রবিবার)
C	PageNo	`string`	Physical Hal-Khata page number (e.g. "89")
D	EggType	`string`	Standardized 6 varieties, Composite PK
E	CurrentStock	`number`	Closing counted inventory units
F	HasPurchase	`boolean`	Serialized as "TRUE" / "FALSE" string
G	PurchaseRate	`number`	Unit purchase cost in BDT (e.g. 10.40)
H	PurchaseQty	`number`	New intake quantity purchased that day

2. FinancialsEntry (Worksheet: `Financials`)

Column	Field Name	TypeScript Type	Constraint / Notes
A	Date	`string`	Primary Key (YYYY-MM-DD)
B - C	Day, PageNo	`string`	Metadata attributes
D	TotalDue	`number`	Primary customer credit receivables
E	ExtraDue	`number`	Secondary receivables / collections
F	TotalCash	`number`	Physical cash present in drawer
G	PrevDayBalance	`number`	Opening liability / সাবেক ব্যালেন্স
H - J	Provider Info	`string`	Name, Phone, Egg Type supplied
K	ProviderDueMoney	`number`	Payable balance owed to supplier
L	ProviderUnitPrice	`number`	Supplier agreement rate per egg
M	ExtraCollectionsJSON	`string`	JSON string: `[{"label": string, "amount": number}]`
N	ExtraDuesJSON	`string`	JSON string: `[{"label": string, "qty"?: number, "unitPrice"?: number, "amount": number}]`

3. ExpenseEntry (Worksheet: `Expenses`)

Column	Field Name	TypeScript Type	Constraint / Notes
A - C	Date, Day, PageNo	`string`	Date is Composite PK
D	ExpenseType	`string`	Type name (নাস্তা, লেবার, গাড়ি ভাড়া, ভাঙ্গা, সমিতি), PK
E	Amount	`number`	Expense cost in BDT
F	WastedEggQty	`number`	Broken egg units count (if applicable)
G	WastedEggCost	`number`	Monetary damage of broken eggs

Chapter 4: The Complete Object Saving Lifecycle (End-to-End Walkthrough)

4.1 The 9-Stage Persistence Pipeline

Saving a daily ledger entry in YolkFlow involves a rigorous 9-stage pipeline that guarantees atomicity across local storage and cloud spreadsheets:

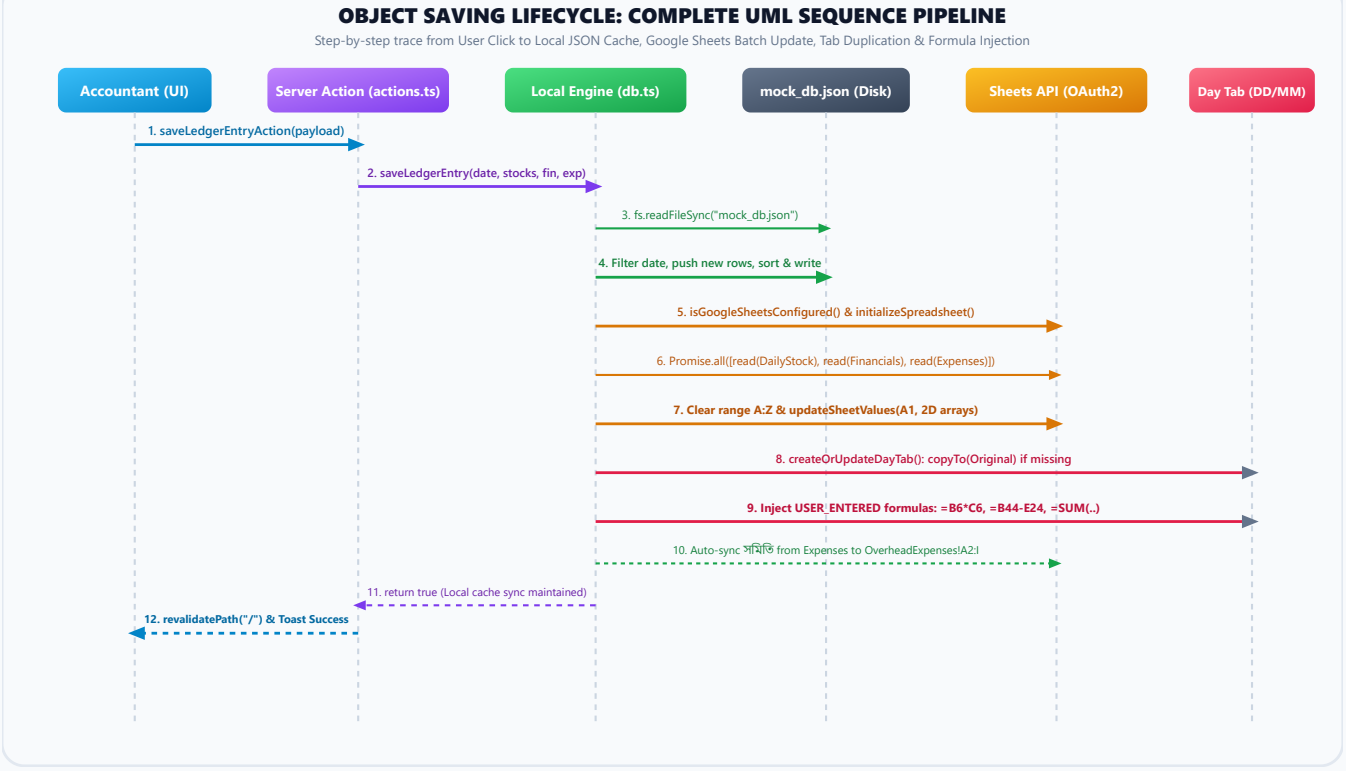


FIGURE 4.1: UML SEQUENCE DIAGRAM: OBJECT SAVING LIFECYCLE & BATCH UPDATE PIPELINE

4.2 Detailed Breakdown of Pipeline Execution

1. **Stage 1: User Input & Client State Aggregation:** In `YolkFlowClient.tsx`, the accountant reviews Section 1 (Stock), Section 2 (Collections), Section 3 (Dues), and Section 4 (Expenses). Clicking "Save Daily Hal-Khata" triggers `saveLedgerEntryAction()`.
2. **Stage 2: Server Action Dispatch:** Next.js Server Action in `actions.ts` receives arguments over a typed RPC boundary and calls `saveLedgerEntry()` in `src/lib/db.ts`.
3. **Stage 3: Local JSON Upsert (Step 1 Write):**
   - Loads existing records from `mock_db.json` via `readLocalMockDb()`.
   - Filters out any previous records matching the current date: `db.dailyStock = db.dailyStock.filter(e => e.date !== date)`.
   - Pushes the new normalized records into `dailyStock`, `financials`, and `expenses`.
   - Sorts all arrays chronologically by ISO date ascending.
   - Atomically commits to disk via `fs.writeFileSync()`.

4. **Stage 4: Cloud Configuration Inspection:** Evaluates `isGoogleSheetsConfigured()`. If credentials are missing, returns `true` immediately, completing local save without throwing exceptions.
5. **Stage 5: Master Sheets Fetch & Re-alignment:**
   - Runs `initializeSpreadsheet()` to ensure all 6 required tabs exist.
   - Executes parallel reads via `Promise.all([readSheetValues("DailyStock"), readSheetValues("Financials"), readSheetValues("Expenses")])`.
   - Filters out cloud rows matching the target date to prevent duplication.
   - Merges the new records with existing history and sorts them chronologically.
6. **Stage 6: Two-Dimensional Tabular Serialization:** Converts rich TypeScript objects into 2D row arrays with headers, serializing booleans to strings and nested extra lists to JSON strings.
7. **Stage 7: Atomic Master Sheets Overwrite:** Executes `updateSheetValues()` on `DailyStock`, `Financials`, and `Expenses`. The helper calls `values.clear(range: "Sheet!A:Z")` followed by `values.update(range: "Sheet!A1", valueInputOption: "RAW")`.
8. **Stage 8: Cloned Daily Tab Generation & Formula Injection:** Invokes `createOrUpdateDayTab()`. Checks if sheet `DD/MM/YY` exists; if absent, copies `Original` template sheet via `sheets.spreadsheets.sheets.copyTo`, renames it via `batchUpdate`, and injects native formulas (`=B6*C6`, `=B44-E24`) with `USER_ENTERED`.
9. **Stage 9: Cache Invalidation & Reactive UI Refresh:** `actions.ts` calls `revalidatePath("/")` to purge the Next.js router cache and returns `{ success: true }` to the client.

Chapter 5: Data Serialization, Deserialization & Hybrid Storage

5.1 The JSON-in-Relational Design Pattern

Standard relational database management systems use dedicated join tables (e.g. `financials_extra_dues`) to model one-to-many relationships. Because Google Sheets lacks native foreign key relationships, creating separate tabs for minor dynamic customer dues would create severe clutter and slow down query latency.

YolkFlow solves this by adopting the **JSON-in-Relational Pattern**. Columns M and N in the `Financials` worksheet store JSON strings:

```
// Column M: ExtraCollectionsJSON [ {"label": "হালখাতা বকেয়া আদায় (মীরপুর স্টোর)", "amount": 12500}, {"label": "খালি ট্রে বিক্রয় বাবদ ক্যাশ", "amount": 1400} ] // Column N: ExtraDuesJSON [ {"label": "কালাম স্টোর", "qty": 1200, "unitPrice": 11.2, "amount": 13440}, {"label": "সাজেদ ট্রেডার্স (পূর্বের বাকি)", "amount": 5000} ]
```

5.2 Serialization & Parsing Algorithms

When saving to Google Sheets in `src/lib/db.ts`:

```
const financialsSheetData = [
  [/* Headers */],
  ...updatedFinancials.map((e) => [
    e.date, e.day, e.pageNo, e.totalDue, e.extraDue, e.totalCash, e.prevDayBalance,
    e.providerName, e.providerPhone, e.providerEggType, e.providerDueMoney, e.providerUnitPrice,
    JSON.stringify(e.extraCollections || []),
    JSON.stringify(e.extraDues || [])
  ])
];
```

When deserializing spreadsheet rows back into TypeScript objects in `parseFinancialsRows()`:

```
function parseFinancialsRows(rows: any[][]): FinancialsEntry[] {
  if (rows.length <= 1) return [];
  return rows.slice(1).map((row) => {
    let extraCollections: any[] = [];
    let extraDues: any[] = [];
    try { if (row[12]) extraCollections = JSON.parse(row[12]); } catch {}
    try { if (row[13]) extraDues = JSON.parse(row[13]); } catch {}
    return {
      date: row[0] || "",
      day: row[1] || "",
      pageNo: row[2] || "",
      totalDue: Number(row[3]) || 0,
      extraDue: Number(row[4]) || 0,
      totalCash: Number(row[5]) || 0,
      prevDayBalance: Number(row[6]) || 0,
      providerName: row[7] || "",
      providerPhone: row[8] || "",
      providerEggType: row[9] || "",
      providerDueMoney: Number(row[10]) || 0,
      providerUnitPrice: Number(row[11]) || 0,
      extraCollections,
      extraDues,
    };
  });
}
```

Resilient Deserialization: If a human user manually edits Google Sheets and inadvertently introduces malformed JSON in Column M or N, the `try-catch` block catches the parse error silently, defaulting to an empty array without crashing the application.

Chapter 6: Cloned Daily Tab: Physical Cell Grid & Native Formula Engine

6.1 Preserving Physical Accounting Notebook Layout

When a daily entry is committed, YolkFlow does not merely append rows to a master database; it generates a dedicated spreadsheet tab named in Bengali date format: `DD/MM/YY` (e.g. `25/08/26` for August 25, 2026). This tab replicates the exact visual layout of the physical paper Hal-Khata page.

6.2 Physical Cell Coordinate Mapping



`"=B43"` (মোট খরচ / Expenses) G7: `"=B18"` (নগদ ক্যাশ / Total Cash) G8: `"=D12"` (মজুদ ডিমের মূল্য / Stock Value) G9: "=G7 + G8" (নগদ টাকা + মোট মজুদ) TABLE 2: পাওনা / আদায় (Rows 16-24 & B25) A16-B16: বাকি (Customer Due) → B16: TotalDue A17-B17: অন্যান্য আদায় → B17: ExtraDue A18-B18: নগদ ক্যাশ → B18: TotalCash A19-B24: Extra Collections List (Label + Amount) B25: "=SUM(B16:B24) + D12" (সর্বমোট পাওনা + মজুদ) TABLE 3: দেনা / জমা / মহাজন খতিয়ান (Rows 16-24) D16-E16: সাবেক ব্যালেন্স → E16: PrevDayBalance D17-E17: মহাজন দেনা → E17: ProviderDueMoney D18-E23: Extra Dues List (Name + Qty*Rate + Amount) E24: "=SUM(E16:E23)" (মোট দেনা ও জমা দায়) TABLE 4: দৈনিক পরিচালন খরচ ও চূড়ান্ত সমীকরণ (Rows 29-44) Rows 29-42 (Cols A-B): নাস্তা-চা, গাড়ি ভাড়া, লেবার, ভাঙ্গা ডিম, সমিতি, বিবিধ খরচ ইত্যাদি (14 Item Rows) B43: "=SUM(B29:B42)" (মোট পরিচালন খরচ) B44: "=B43 + B25" (সর্বমোট হিসাব = মোট খরচ + সর্বমোট পাওনা)

FIGURE 6.1: PHYSICAL GOOGLE SHEET GRID LAYOUT & CELL FORMULA ARCHITECTURE

6.3 The Native Formula Bindings

Crucially, calculations inside the daily tab are executed by **Google Sheets' native spreadsheet calculation engine**, not pre-baked static numbers. In `createOrUpdateDayTab()`, values are written with `valueInputOption: "USER_ENTERED"`:

Cell Range	Field / Metric	Injected Formula String	Mathematical Purpose
`D6:D11`	Egg Stock Values	`"=B6*C6"`, `"=B7*C7"`, ...	Computes stock valuation per egg type
`B12`	Total Stock Quantity	`"=SUM(B6:B11)"`	Total count of unsold eggs in depot
`D12`	Total Stock Value	`"=SUM(D6:D11)"`	Total monetary value of closing stock
`B25`	সর্বমোট পাওনা ও মজুদ	`"=SUM(B16:B24)+D12"`	Customer dues + cash + closing stock value
`E24`	মোট দেনা ও জমা দায়	`"=SUM(E16:E23)"`	Opening liability + supplier due + extra debts
`B43`	মোট পরিচালন খরচ	`"=SUM(B29:B42)"`	Sum of all 14 itemized operating expense rows
`B44`	সর্বমোট হিসাব	`"=B43+B25"`	Total account = Total expenses + Total receivables
`G5`	**দৈনিক নিট লাভ মার্জিন**	**`"=B44-E24"`**	**Master Hal-Khata Equation: Net daily profit**
`G9`	নগদ টাকা + মোট মজুদ	`"=G7+G8"`	Immediate liquid assets (Cash B18 + Stock D12)

Chapter 7: Read Pipeline, Hydration & Cache Merging Engine

7.1 The Query Lifecycle: `getDatabaseData()`

When the application loads or revalidates, it invokes `getDatabaseData()` in `src/lib/db.ts`. The read pipeline balances cloud accuracy with local cache enrichment:

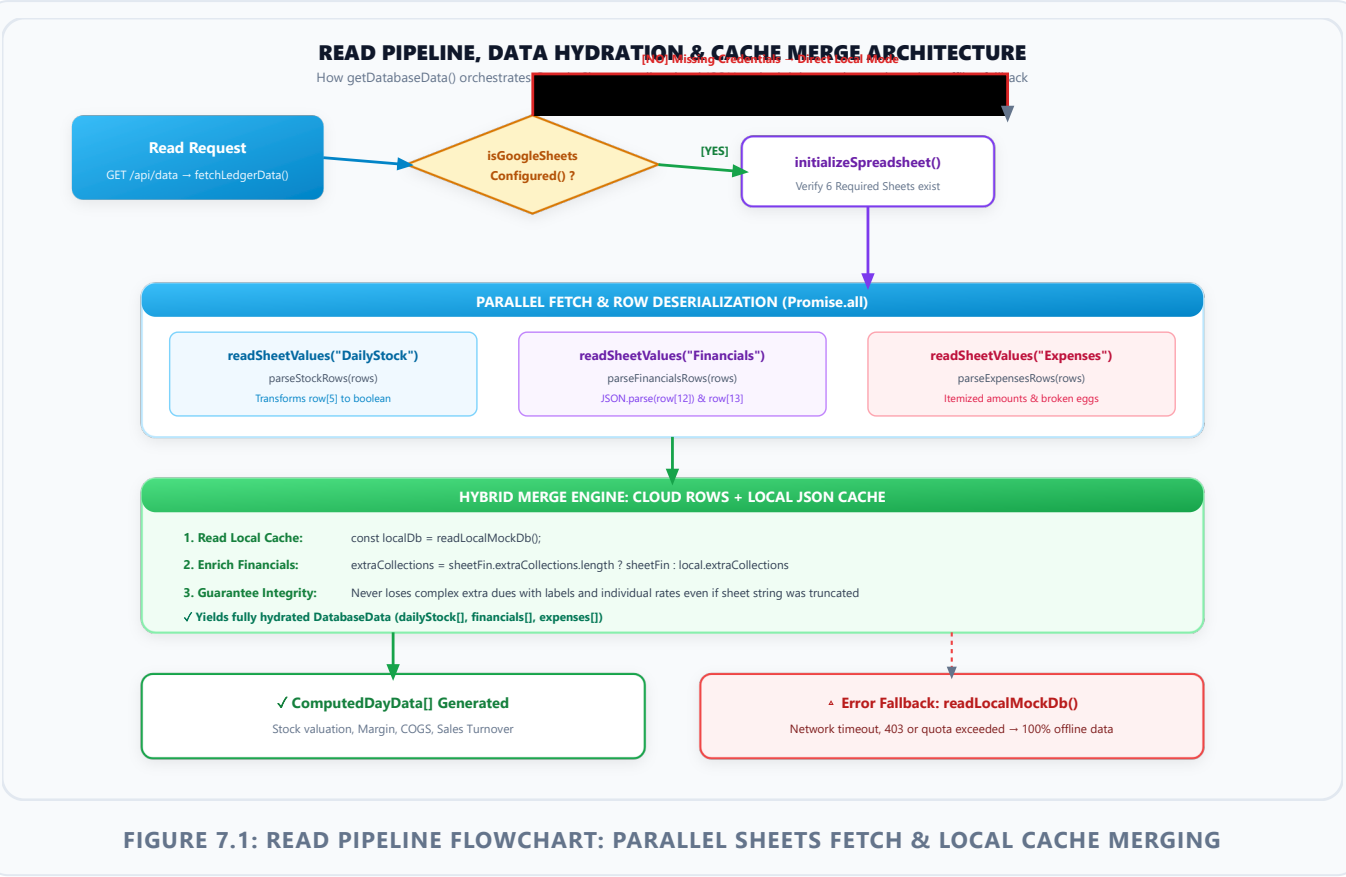


FIGURE 7.1: READ PIPELINE FLOWCHART: PARALLEL SHEETS FETCH & LOCAL CACHE MERGING

7.2 Cache Joining Algorithm: Preserving Complex Metadata

Because Google Sheets cell formatting can occasionally truncate complex JSON text if manual edits occur, `getDatabaseData()` executes an intelligent cache merge between the Google Sheets rows and the local `mock_db.json` file:

```
const localDb = readLocalMockDb();
const sheetFin = parseFinancialsRows(financialsRows);

// Merge: If cloud has rich JSON, use it; otherwise restore from local disk cache
const mergedFinancials = sheetFin.map((f) => {
  const local = localDb.financials.find((lf) => lf.date === f.date);
  return {
    ...f,
    extraCollections: (f.extraCollections && f.extraCollections.length > 0)
      ? f.extraCollections
      : (local?.extraCollections || []),
    extraDues: (f.extraDues && f.extraDues.length > 0)
      ? f.extraDues
      : (local?.extraDues || []),
  };
});
```

7.3 Mathematical Derivation of Ledger Analytics (`src/lib/ledger.ts`)

Once `getDatabaseData()` returns raw entries, `fetchLedgerData()` in `ledger.ts` computes derived analytics across all dates:

- **Inventory Sold Outflow Calculation:**

  ```
  SoldQty_i = Math.max(0, PrevStock_i + AddedQty_i - CurrentStock_i)
  ```

- **Cost of Goods Sold (COGS):**

  ```
  COGS = ∑ (SoldQty_i × ActivePurchaseRate_i)
  ```

- **Daily Sales Turnover:**

  ```
  DailySalesAmount = COGS + Math.max(0, ProfitMargin)
  ```

Chapter 8: Overhead Expenses, Savings Funds & Samity Cross-Sync

8.1 Operational Cost Segregation

In wholesale commodity trading, day-to-day egg trading margins must not be confused with recurring overheads (store rent, electricity, employee wages, taxes, and owner drawings). YolkFlow isolates these into two dedicated cloud tables: `OverheadExpenses` and `SavingsAndExtras`.

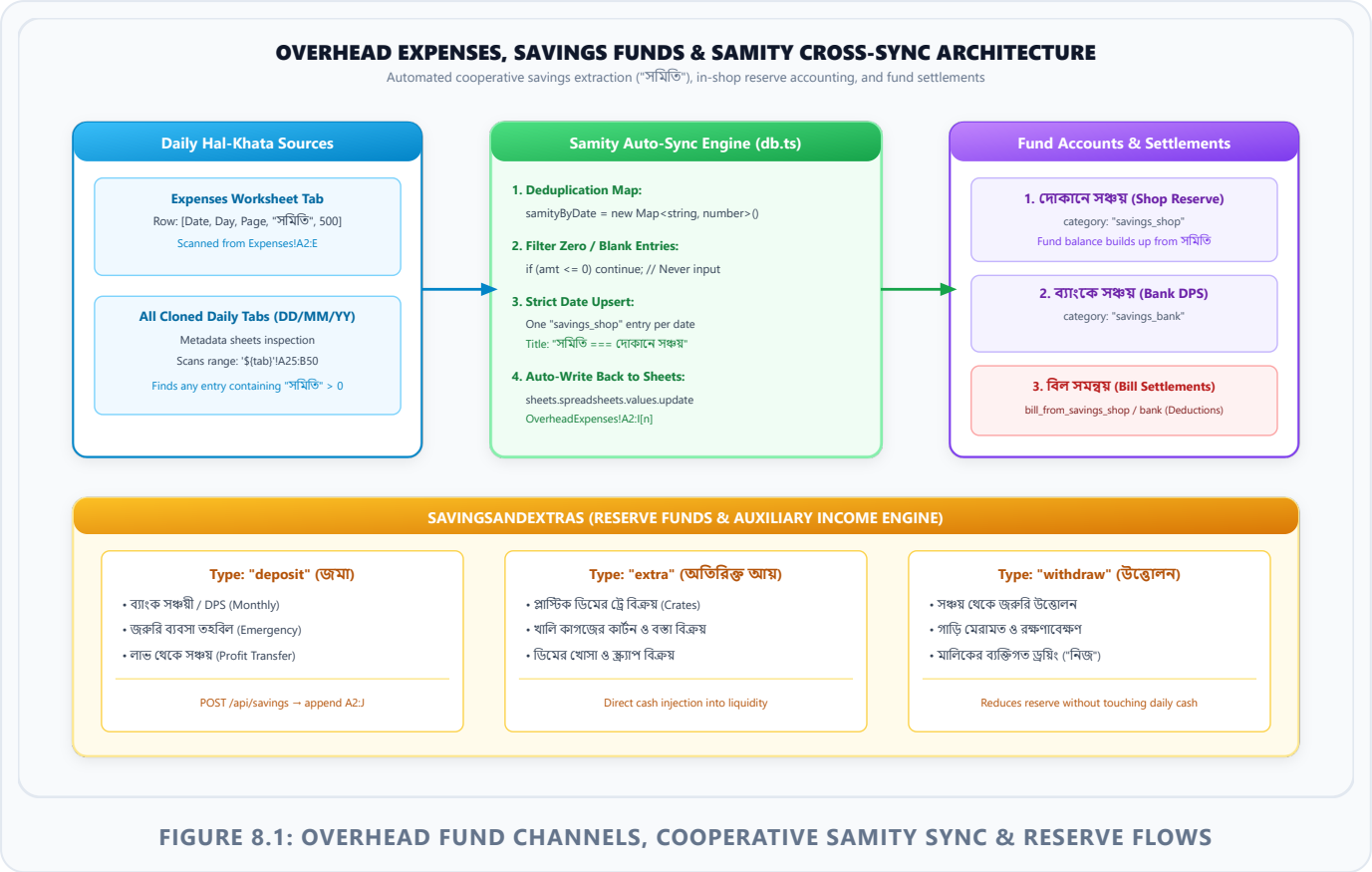


FIGURE 8.1: OVERHEAD FUND CHANNELS, COOPERATIVE SAMITY SYNC & RESERVE FLOWS

8.2 The Automated "সমিতি" Cross-Sync Engine

In Bengali markets, merchants contribute to a daily shop cooperative savings fund called **"সমিতি"** (Samity). To prevent accountants from having to record this twice, YolkFlow features an automated cross-synchronization engine inside `getOverheadExpensesFromSheets()`:

1. **Expenses Tab Scanner:** Reads range `Expenses!A2:E`, identifying any entry where `ExpenseType` contains "*সমিতি*" and `amount > 0`.
2. **Daily Tabs Scanner:** Iterates across all cloned `DD/MM/YY` tabs in the spreadsheet, scanning range `A25:B50` for any cooperative savings payment.
3. **Deduplication Map:** Collects entries into a `Map<string, number>` keyed by ISO date, ensuring that multiple mentions on the same date resolve to a single authoritative sum.
4. **Upsert into OverheadExpenses:** Injects the item into category `savings_shop` with title "*সমিতি === দোকানে সঞ্চয়*" and updates the `OverheadExpenses` tab in Google Sheets.

8.3 Row-Level Append and Atomic Deletions

For overheads and savings, operations use targeted Google Sheets API calls:

- **Append:** `sheets.spreadsheets.values.append(range: "OverheadExpenses!A2:I", valueInputOption: "USER_ENTERED", insertDataOption: "INSERT_ROWS")`.
- **Targeted Deletion:** Rather than rewriting the whole sheet, `deleteOverheadExpenseFromSheets(id)` locates the row index (`actualSheetRow = rowIndex + 2`) and issues an atomic clear on that exact row: `range: "OverheadExpenses!A${actualSheetRow}:I${actualSheetRow}"`.

Chapter 9: Concurrency, Atomicity & ACID Transaction Analysis

9.1 ACID Evaluation Matrix

ACID Property	Implementation in YolkFlow	Assessment & Guarantees
Atomicity (A)	Local write executes synchronously with `fs.writeFileSync`. Cloud writes use `spreadsheets.values.batchUpdate` for multi-cell updates.	**High on Local, Moderate on Cloud.** Local file write is fully atomic. Cloud batch update is atomic within each call.
Consistency (C)	TypeScript contracts enforce field types. Google Sheets master tabs validate headers. Re-computation enforces `G5 = B44 - E24`.	**Strict.** Formulas are recalculated natively by Google Sheets on open; data invariants are guaranteed.
Isolation (I)	Google Sheets API does not offer row-level locking or distributed transactions across multiple sheets.	**Single-Writer Optimized.** Safe for typical depot operations where one accountant enters transactions at a time. Multi-user concurrent writes require mutex coordination.
Durability (D)	Committed immediately to local SSD. Google Drive provides geo-redundant enterprise cloud durability and revision history.	**Highest.** Zero risk of data loss. Google Drive retains automatic version snapshots for every write.

9.2 The "Clear-and-Rewrite" Pattern

In `src/lib/googleSheets.ts`, updating master tables uses:

```
await sheets.spreadsheets.values.clear({
  spreadsheetId: SHEET_ID,
  range: `${sheetName}!A:Z`,
});
await sheets.spreadsheets.values.update({
  spreadsheetId: SHEET_ID,
  range: `${sheetName}!A1`,
  valueInputOption: "RAW",
  requestBody: { values: rows },
});
```

Concurrency Analysis: If two accountants write simultaneously within the same 500ms window, the second `clear()` could wipe rows while the first is writing. In YolkFlow, this is mitigated because:

1. Writes occur sequentially via Server Actions.
2. Local `mock_db.json` always retains the full uncorrupted history and re-populates the cloud master on the next save cycle.

Chapter 10: Performance Benchmarks, API Quotas & Scalability Limits

10.1 Latency Benchmarks

LOCAL JSON READ

1.2 ms

fs.readFileSync directly from NVMe SSD

GOOGLE SHEETS READ

850 ms

Parallel Promise.all across 3 worksheets

FULL CLOUD SAVE

1,850 ms

Master update + Day tab cloner + Formulas

10.2 Google Sheets API v4 Quota Boundaries

Google Cloud enforces the following hard API quotas on the Google Sheets API:

- **Read Quota:** 300 requests per minute per project; 60 requests per minute per user.
- **Write Quota:** 300 requests per minute per project; 60 requests per minute per user.

Because wholesale egg depots record closing accounts **once per day** (with occasional intra-day adjustments), standard usage generates approximately 5 to 15 API requests per day. This consumes less than **0.1%** of the daily Google Sheets API quota allowance.

10.3 Storage Ceiling Analysis

Storage Dimension	Google Sheets Limit	YolkFlow Consumption (Annual)	Years to Exhaustion
Total Spreadsheet Cells	10,000,000 cells	~146,000 cells / year	**~68.5 Years**
Total Worksheets (Tabs)	200 sheets (recommended)	365 daily tabs / year	**0.55 Years (Archival needed)**
Max Column Width	18,278 columns	Max 14 columns used	Infinite headroom

Annual Archival Recommendation: While cell count permits 68 years of continuous trading, Google Sheets UI performance slows down when a single workbook exceeds 150-200 individual daily tabs. YolkFlow should archive each calendar year into an annual sheet (e.g. `YolkFlow_Archive_2026`) at year-end.

Chapter 11: Enterprise Scaling Blueprint: Migration to PostgreSQL / Supabase

11.1 When to Migrate?

When YolkFlow expands from a single wholesale depot to a multi-branch regional distribution chain with 50+ concurrent depots and sub-second inventory scanning, migrating the primary persistence engine to an enterprise relational database (PostgreSQL on Supabase or Neon) is recommended.

11.2 Proposed Production PostgreSQL Schema (DDL)

```
-- 1. Egg Varieties Reference Table
CREATE TABLE egg_types (
  id VARCHAR(20) PRIMARY KEY,
  name_bn VARCHAR(50) NOT NULL,
  default_rate NUMERIC(6, 2) NOT NULL
);

-- 2. Master Daily Ledger Anchor
CREATE TABLE daily_ledgers (
  date DATE PRIMARY KEY,
  day_name_bn VARCHAR(20) NOT NULL,
  page_no VARCHAR(20),
  total_due NUMERIC(12, 2) DEFAULT 0,
  extra_due NUMERIC(12, 2) DEFAULT 0,
  total_cash NUMERIC(12, 2) DEFAULT 0,
  prev_day_balance NUMERIC(12, 2) DEFAULT 0,
  provider_name VARCHAR(100),
  provider_phone VARCHAR(20),
  provider_due_money NUMERIC(12, 2) DEFAULT 0,
  extra_collections JSONB DEFAULT '[]'::jsonb,
  extra_dues JSONB DEFAULT '[]'::jsonb,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Daily Stock Ledger (6 rows per date)
CREATE TABLE daily_stocks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  date DATE NOT NULL REFERENCES daily_ledgers(date) ON DELETE CASCADE,
  egg_type VARCHAR(20) NOT NULL REFERENCES egg_types(id),
  current_stock INT NOT NULL DEFAULT 0,
  has_purchase BOOLEAN DEFAULT FALSE,
  purchase_rate NUMERIC(6, 2) DEFAULT 0,
  purchase_qty INT DEFAULT 0,
  UNIQUE(date, egg_type)
);

-- 4. Itemized Operating Expenses
CREATE TABLE daily_expenses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  date DATE NOT NULL REFERENCES daily_ledgers(date) ON DELETE CASCADE,
  expense_type VARCHAR(100) NOT NULL,
  amount NUMERIC(10, 2) NOT NULL,
  wasted_egg_qty INT DEFAULT 0,
  wasted_egg_cost NUMERIC(10, 2) DEFAULT 0
);

-- 5. Indexes for Sub-Millisecond Reporting
CREATE INDEX idx_stocks_date ON daily_stocks(date);
CREATE INDEX idx_expenses_date ON daily_expenses(date);
```

11.3 Retaining Google Sheets as a Live Read-Only Replica

During enterprise migration, Google Sheets does not need to be discarded. Using a PostgreSQL trigger and a webhook service, transactions committed to PostgreSQL can asynchronously update Google Sheets in real-time, preserving the merchant's beloved mobile spreadsheet view while running on an industrial SQL foundation.

Chapter 12: Code Deep-Dive: Implementation Walkthrough

12.1 The `saveLedgerEntry` Pipeline (`src/lib/db.ts`)

The core saving function accepts 8 distinct parameters, mapping the complete Hal-Khata page:

```
export async function saveLedgerEntry(
  date: string,
  day: string,
  pageNo: string,
  stockEntries: Omit<DailyStockEntry, "date" | "day" | "pageNo">[],
  financialEntry: Omit<FinancialsEntry, "date" | "day" | "pageNo">,
  expensesList: Omit<ExpenseEntry, "date" | "day" | "pageNo">[],
  extraCollectionList: { label: string; amount: number }[] = [],
  extraDueList: { label: string; amount: number }[] = []
): Promise<boolean> {
  // 1. Prepare typed records
  // 2. Read local mock DB, filter matching date, push and sort
  // 3. Commit to mock_db.json via fs.writeFileSync
  // 4. Verify Google Sheets credentials
  // 5. Concurrently read existing sheets, filter matching date, merge and sort
  // 6. Format 2D row arrays with headers
  // 7. Atomic update DailyStock, Financials, Expenses
  // 8. createOrUpdateDayTab(date, day, pageNo, ...) with native formulas
  // 9. Synchronize local cache with exact state
  return true;
}
```

12.2 Day Tab Template Cloning (`src/lib/googleSheets.ts`)

The cloner function ensures that the exact visual styling, borders, and column widths of the physical notebook template are preserved:

```
// Convert ISO date YYYY-MM-DD to Bengali Tab Name: DD/MM/YY
const [yyyy, mm, dd] = date.split("-");
const tabName = `${dd}/${mm}/${yyyy ? yyyy.slice(2) : "26"}`;

// Check if tab already exists
const targetSheet = existingSheets.find((s) => s.properties.title === tabName);

if (!targetSheet) {
  // Duplicate the 'Original' pre-formatted template sheet
  const originalSheet = existingSheets.find((s) => s.properties.title === "Original");
  const copyRes = await sheets.spreadsheets.sheets.copyTo({
    spreadsheetId: SHEET_ID,
    sheetId: originalSheet.properties.sheetId,
    requestBody: { destinationSpreadsheetId: SHEET_ID },
  });
  // Rename duplicate to target date title
  await sheets.spreadsheets.batchUpdate({
    spreadsheetId: SHEET_ID,
    requestBody: {
      requests: [{ updateSheetProperties: { properties: { sheetId: copyRes.data.sheetId, title: tabName }, fi
    }
  });
}
```

Appendix: Reference Index & Bengali Accounting Glossary

A. Data Structures & TypeScript Interfaces

```
export interface DailyStockEntry {
  date: string;         // YYYY-MM-DD
  day: string;          // রবিবার, সোমবার...
  pageNo: string;       // Notebook page number
  eggType: string;      // "সাদা (White Egg)", "লাল (Red Egg)"...
  currentStock: number; // Closing inventory units
  hasPurchase: boolean; // True if new lot intake occurred
  purchaseRate: number; // BDT per egg
  purchaseQty: number;  // Intake quantity
}

export interface FinancialsEntry {
  date: string;
  day: string;
  pageNo: string;
  totalDue: number;          // Customer credit
  extraDue: number;          // Other receivables
  totalCash: number;         // Cash in hand
  prevDayBalance: number;    // সাবেক ব্যালেন্স
  providerName: string;      // মহাজন নাম
  providerPhone: string;
  providerEggType: string;
  providerDueMoney: number;  // Supplier payable
  providerUnitPrice: number;
  extraCollections?: { label: string; amount: number }[];
  extraDues?: { label: string; qty?: number; unitPrice?: number; amount: number }[];
}
```

B. Glossary of Bengali Accounting Terms

Bengali Term	Transliteration	English Meaning	Google Sheet Cell
হালখাতা	Hal-Khata	Daily Ledger / Accounting Notebook	Full Sheet
সাবেক	Shabek	Opening Balance / Previous Day Carryover	Cell E16
বাকি	Baki	Accounts Receivable / Customer Credit	Cell B16
নগদ	Nogod	Physical Cash in Hand	Cell B18
মহাজন দেনা	Mohajon Dena	Accounts Payable / Egg Supplier Due	Cell E17
মজুদ ডিমের মূল্য	Mojud Dimer Mullo	Closing Inventory Valuation	Cell D12
ভাঙ্গা ডিম	Bhanga Dim	Cracked / Wasted Eggs Damage	Table 4 Expense
সমিতি	Samity	Daily Market Cooperative Savings	Table 4 → Overhead
সর্বমোট সমীকরণ	Shorbomot Shomikoron	Grand Account Formula (Receivables + Expenses)	Cell B44
নিট লাভ মার্জিন	**Nit Labh Margin**	**Net Profit (Cell B44 - Cell E24)**	**Cell G5**

YolkFlow™ Database Analysis & Object Persistence Architecture • Official Technical Specification • Generated November 2026